Humans made the biggest decisions in the production of the code. Such as the main goals stack and architecture. In the context.md the AI model is told to stop and ask rather than go beyond simple instructions. AI asked questions before even trying to modify any code and then was reviewed before implantation. That being said, here is a more detailed explanation of what exactly AI tools were used for as we progressed.

AI tools used running inside the IDE were used mostly over the course of this section.

Tool	Model	Evidence
Claude Code (IDE)	Claude Opus 5.5	Commit trailers on 5 commits (1–4 Oct 2026); Claude Code session logs (30 Sep – 9 Oct 2026)
Claude Code (IDE)	Claude Sonnet 5	Commit trailer on ded95aa (15 Sep 2026); planning-session screenshot
Gemini CLI	Did not use this agent is mentioned in the context.MD fille.	Named as a permitted agent in context.md; no commit or file shows it was used

Here are the co authored commits showing the model used as well as an explanation of what it did.

Commit	Date	Model	What it did
840003d	2026-10-04	Opus 5.5	Merged the Nick2 branch (teams, roles) into the Kaleb branch
b45a45c	2026-10-04	Opus 5.5	Removed committed server keys; added them to .gitignore
d5456d9	2026-10-03	Opus 5.5	Added docstrings and comments to every module; wrote docs/MERGING.md
f6eda1e	2026-10-03	Opus 5.5	Message attachment UI, client crash and Retry fix, tests to 100% coverage

50687bb	2026-10-01	Opus 5.5	Team and admin screens, client robustness, concurrency fixes; removed goals 4–7 code
ded95aa	2026-09-15	Sonnet 5	Fixed the initial scaffold to track source files, not build artifacts

We have also listed AI assisted areas of the codebase. That details most written or AI modified work.

Component	AI-written or AI-modified work
client/views, client/viewmodels	Login screen, Teams and Users screens, permissions, attachment UI, background request handling, session events
client/main_thread_gc.py, client/api_client	Crash fix (main-thread garbage collection), Retry logic, API error handling
server/api, server/services	Auth and sessions, users and teams routes, login throttling, error handling, database session locking
server/models, server/schemas, shared/	Account, message and role rules; team-lead changes; removal of project/task/dashboard code (goals 4–7)
tests/	Auth, security, concurrency, robustness, edge-case, seed and GUI tests, bringing coverage to 100%

Comments and docs	Docstrings across 55 files and docs/MERGING.md
Repo hygiene	Removing leaked keys from git; .gitignore updates; branch merge and conflict resolution

Examples of how the AI was used and the tasks that were being completed at the time of the questions.

Kind of task	Examples from the session logs
Build features from the goals list	"Let's do goal number 1", "Log in screen", "Build screens for 1–3", "Add button to GUI for messages to share files"
Testing and coverage	"Run tests… focusing on everything including past features", "Do coverage test", "Yes, looking for 100%"
Security review	"Stress test every security aspect", then fix what was found
Bug fixing	Duplicate message send, client crash and Retry, connection timeouts over VPN

Clean-up	Removing unused files from an earlier AI session; deleting committed server keys
Documentation	"Make comments… so that someone can understand what everything does" and mark merge-critical code
Git operations	Branch comparison, merging Nick2 into Kaleb, conflict resolution, local backups, commits and pushes
Learning and explanation	Line-by-line code walkthroughs, "how do APIs work", explanations for a beginner
Running the app	Starting the server and client, checking test accounts, troubleshooting connections

Testing & Coverage

- The project's automated suite (474 tests across the server, client, and shared packages, ~93% coverage) was grown and reviewed with AI assistance over several passes:
- Closed a genuine gap found this way: server/email_sender.py's real SMTP-sending code was previously untested (35% -> 100%), by writing tests against the real function rather than its test-suite stub.
- Ran a dedicated multi-agent code-review pass (ten parallel analyses of the 2FA feature's diff) and synthesized the results into seven deduplicated, severity-ranked findings, the two most important being a missing account-recovery path for 2FA-locked-out users and

an orphaned-database-row risk on email failures, rather than reporting every raw finding uncritically.
- When new bugs were found during a later coverage pass (the conversation-leave permission bug and the _error_message crash risk, both above), wrote the regression tests alongside the fixes in the same pass.

Git, Merge & Operational Workflow

- Beyond code, AI assistance was used for day-to-day development operations, always stopping to confirm before anything destructive or hard to reverse.
- Merge conflict resolution reconciled a messy, multi-layered conflict in README.md (nested conflict markers left over from an earlier, improperly-resolved commit) during an interactive rebase.
- Recognized and aborted a risky rebase. A 39-commit interactive rebase hit a 13-file conflict that would have required manually reconstructing an auth-system merge the team had already done correctly via a tested merge commit earlier the same day. Rather than attempt that reconstruction by hand inside conflict markers, flagged the risk, asked the user how to proceed, and aborted, restoring the branch to its known-good state with nothing lost.
- Environment debugging: diagnosed a server unreachable over Tailscale as a client-side http:// vs https:// scheme bug rather than a network issue; diagnosed a "server took too long to answer" error down to a stray Docker container that had permanently claimed port 8000 on the host machine; diagnosed a ModuleNotFoundError down to the system Python being invoked instead of the project's virtualenv.
- Database hygiene: repeatedly backed up the dev database before any destructive operation, used transaction-safe bulk deletes to avoid crashing on legacy encrypted columns, and always verified foreign-key-safe deletion order before removing test accounts.

Github Pull Request:
https://github.com/Ktang2001/Kairos/commit/fb03e512bdcbcb789fccccc709e273408387a3bc